Persona Recruiter Interview Master Sheet

Use this as your preparation document rather than memorizing every sentence. Keep your spoken answers conversational and generally under 90 seconds.

One important rule: **never add technical details that you cannot defend in a follow-up question.** For metrics such as “68% improvement” or “one million transactions,” know exactly how they were measured.

---

1. Detailed company description

What Persona does

Persona is a **B2B identity infrastructure company** that helps organizations verify real people and real businesses online. Its broader goal is to make online interactions safer while protecting legitimate users’ privacy.

Companies use Persona when they need to answer questions such as:

- Is this user a real person?
- Does this government ID appear authentic?
- Is the person old enough to access a service?
- Is this business legitimate?
- Are several accounts connected to the same fraud network?
- Does this customer satisfy KYC, KYB or AML requirements?
- Should an account be approved, rejected or manually reviewed?

Persona describes its mission as **“keeping the internet human”** and says it is building the identity layer for a safer and more accessible internet.

A simple interview explanation is:

> Persona provides the infrastructure that companies use to verify identities, prevent fraud and manage identity-related decisions throughout a user’s lifecycle.

The problem Persona solves

Traditionally, companies had to connect separate vendors for:

- Government-ID verification
- Selfie and biometric checks
- Phone and email risk
- Business verification
- Watchlist screening
- Fraud detection
- Manual review
- Compliance workflows

That creates fragmented systems, duplicated data, inconsistent user experiences and significant operational work.

Persona's approach is to provide these capabilities through a **configurable platform**. Customers can select the verification methods, risk signals, decision rules and review processes appropriate for their industry.

This matters because identity requirements differ greatly. A bank may need strict KYC and AML checks, while a marketplace may care more about account fraud, and a social platform may need age assurance.

Persona's major products

1. Verifications

Verifications determine whether identity information is genuine.

Persona supports methods such as:

- Government-ID verification
- Document AI
- Selfie recognition
- Selfie-based age estimation
- Database verification
- Phone and email verification
- Mobile driver's licenses
- NFC-based checks

The platform is designed to combine authoritative records, documents and modern fraud signals rather than relying on one method alone.

2. Dynamic Flow

Dynamic Flow allows customers to build customized identity-collection experiences.

For example, a company could create a flow that:

1. Requests a phone number.
2. Evaluates risk signals.
3. Requests a government ID only when necessary.
4. Escalates suspicious users to a selfie check.
5. Sends uncertain cases to manual review.

This can improve conversion because low-risk customers do not necessarily have to complete every possible check.

3. Workflows

Workflows automate identity decisions and operational processes.

They can be used for:

- KYC
- KYB
- Workforce verification
- Reverification
- Fraud escalation
- Manual-review routing
- Compliance processes

The main value is that customers can define rules and actions instead of manually processing every verification result.

4. Graph

Graph helps detect **coordinated fraud**.

A single account may appear normal, but fraud becomes visible when relationships are analyzed. Several accounts might share:

- The same device
- The same address
- Similar identity documents
- The same phone number
- Related payment instruments
- Common behavioral signals

Graph performs link analysis to identify these connections and expose fraud networks that an account-by-account review could miss.

5. Cases

Cases provides an investigation and review environment.

Operations or compliance teams can:

- Review verification results
- Investigate suspicious activity
- Examine supporting evidence
- Record decisions
- Escalate cases
- Use Persona's AI-assisted review tools

It connects automated fraud detection with human decision-making.

6. Relay

Relay is Persona's privacy-focused eligibility-verification product.

It can verify a specific claim—such as whether someone is over 18 or is a real human—without sharing the person's full identity information with the organization requesting the proof. Persona describes it as a way to verify eligibility without the requesting organization seeing or storing unnecessary personally identifiable information.

Customers and industries

Persona displays customers including Square, LinkedIn, OpenAI, Twilio, Etsy, Lyft, Reddit, Coursera, Figma, Gusto, Brex and others.

Its platform serves industries including:

- Financial technology
- Payments
- Marketplaces
- Cryptocurrency
- Digital health
- Government
- Financial institutions
- E-learning and higher education

The common requirement across these industries is **online trust**: determining whether a person or business is legitimate while minimizing friction for genuine users.

---

2. Company history and leadership

Persona was founded by **Rick Song and Charles Yeh**. Rick Song is Persona's co-founder and CEO and previously worked on identity, fraud and risk products at Square. Charles Yeh previously worked at Dropbox.

According to First Round's 2026 interview with Rick Song, Persona raised a **$200 million Series D at a $2 billion valuation**. The company's original vision was broader than building one ID-verification tool—it aimed to build a horizontal identity platform that could support many industries and use cases.

A useful insight from Persona's history is that the founders intentionally avoided becoming a narrowly vertical identity vendor. They worked with different industries and attempted to generalize common identity problems into configurable infrastructure.

That platform mindset is relevant to the residency because the infrastructure team is supporting systems used across many products, customers, compliance environments and traffic patterns.

---

3. Persona's values and culture

Persona publicly highlights three values:

Lead by serving

Leadership is not only about authority. It means supporting customers and teammates, removing obstacles and helping others succeed.

For your interview, connect this with:

- Writing documentation
- Improving CI/CD for other developers
- Participating in code reviews
- Building systems that reduce manual work
- Improving observability so others can troubleshoot faster

People first

Persona emphasizes the effect its technology and decisions have on actual people.

For an identity company, this matters because false rejection can block a legitimate person from accessing employment, financial services, education or an online platform.

Connect this with your focus on:

- Reliability
- User experience
- Responsible system design
- Clear communication
- Avoiding unnecessary operational friction

Work in public

This generally means sharing context, decisions, progress and lessons rather than allowing knowledge to remain isolated.

Connect this with:

- Documenting deployments
- Writing clear runbooks
- Sharing production findings
- Asking for early feedback
- Participating in code reviews
- Making system behavior observable

Persona describes its mission as building a more human internet and lists “lead by serving,” “people first” and “work in public” among its principles.

Persona’s early-career engineering material also emphasizes shipping meaningful work early, collaborating across engineering, product and design, and receiving mentorship from engineers with experience at companies such as Square, Dropbox and Coursera.

---

4. Recent Persona developments

These are valuable to mention because they show that you researched beyond the basic company description.

Faster fraud-graph investigation

Persona announced that Graph can now run complex queries **four times faster**. It also added capabilities such as custom nodes, image similarity and workflow actions based on graph findings.

Why this matters for infrastructure

Graph-based investigation can involve computationally expensive relationship queries. Supporting it requires:

- Efficient data storage
- Fast graph queries
- Scalable compute
- Reliable ingestion pipelines
- Observability
- Controlled latency

You can connect this to your database optimization, ETL and performance experience.

Detection of AI-generated and manipulated documents

Persona's Document AI now uses hundreds of indicators to identify:

- Visually modified documents
- Suspicious metadata
- Editing-software usage
- Pixel-level anomalies
- AI-generated textures and structures

This addresses a growing problem: generative AI makes fraudulent identity documents easier to produce.

Infrastructure connection

These checks may require image-processing pipelines, asynchronous computation, model serving and large-scale data processing. The infrastructure must handle this without creating excessive onboarding latency.

Passive fraud detection

Persona introduced tools that collect passive signals throughout the user lifecycle, not only during the initial verification.

That reflects an important identity-security concept:

> Verification is not necessarily a one-time event. Risk may change after the user is onboarded.

Persona also described compute-heavy escalated verifications that run asynchronously so more sophisticated fraud checks do not unnecessarily block onboarding.

Privacy-preserving verification through Relay

Relay allows a company to verify a claim such as “this person is over 18” without receiving the person’s full identity data.

This is important because identity companies must balance two goals:

- Establish sufficient trust
- Avoid collecting or exposing unnecessary personal information

Persona positions privacy as a foundational part of its platform.

AI-assisted case review

Persona announced Case Review Agents that learn from a customer team’s previous decisions and provide recommendations with reasoning for incoming cases.

The goal is to reduce manual-review workload while keeping decision context visible to human investigators.

Configuration as code

Persona is introducing the ability to manage templates and workflows as code inside a repository.

Changes can go through:

- Version control
- Code review
- Testing
- Audit trails
- Rollback procedures

This is particularly relevant to your background because it brings software-engineering and CI/CD practices into identity-operations configuration.

A strong line for the recruiter:

> I was especially interested in Persona's move toward configuration as code because it applies the same testing, review and rollback discipline that engineers expect from application code to identity workflows.

Platform integrations and AI assistant

Persona says its integration library has expanded to more than 75 integrations, including connections with systems such as Workday, Microsoft Entra and Zoom.

It has also introduced Persona Assistant, which can help users navigate the dashboard, locate records, answer data questions and configure Persona through a conversational interface.

Government and digital-ID expansion

Recent releases include:

- Support for California mobile driver's licenses
- Real-time California DMV record lookups
- Expanded international digital-ID support
- NFC-based identity capabilities
- FedRAMP Moderate authorization for federal-government use cases

These developments indicate that Persona is expanding across both commercial and regulated public-sector environments.

---

5. Why infrastructure is critical at Persona

Persona's infrastructure organization owns or supports:

- Cloud platforms
- Kubernetes
- Databases
- CI/CD
- Networking
- Observability
- Production reliability
- Performance

Residents are expected to work on real production systems rather than tutorial-only assignments.

Infrastructure is especially important at Persona because:

Reliability affects real users

If the platform is unavailable, customers may be unable to onboard or verify legitimate users.

Latency affects conversion

If verification takes too long, genuine users may abandon the process.

Security and privacy are fundamental

The platform works with sensitive identity information, so access control, encryption, auditing and data-handling practices are essential.

Fraud workloads are unpredictable

Fraud attacks and major customer events can produce sudden traffic or computational spikes.

Customers operate globally

The infrastructure may need to support different data requirements, regulations, integrations and identity types.

---

6. Interview-ready company answer

"What do you know about Persona?"

From my research, Persona is a B2B identity infrastructure company that helps organizations verify real people and businesses, prevent fraud and manage identity-related processes such as KYC, KYB and age assurance.

What differentiates Persona is that it is not only a government-ID verification API. It provides a configurable platform containing Verifications, Dynamic Flow, Workflows, Graph and Cases. That allows customers to design different verification experiences, automate decisions, identify connected fraud and manage investigations throughout the user lifecycle.

I also looked into Persona's recent releases. The company has improved Graph query performance, expanded Document AI to detect manipulated and AI-generated documents, introduced privacy-preserving verification through Relay and is adding AI-assisted case review and configuration-as-code capabilities.

What makes this particularly interesting to me is that identity is a reliability-, security- and latency-sensitive product. The infrastructure directly affects customer trust and whether legitimate users can access important services.

---

7. Your main introduction

Hi, I'm Sahaj. I recently completed my Master's in Computer Science at USC, and I have over two years of experience building production backend and cloud-based systems using Python, PostgreSQL, AWS, Azure, Docker, Kubernetes, and CI/CD.

Most recently, at USC Dornsife, I developed FastAPI and PostgreSQL services supporting more than 500 users. I worked across the full production lifecycle, including automated deployments, validation, structured logging, monitoring, and performance optimization. One of my key contributions was helping reduce system latency from approximately 1.2 seconds to 350 milliseconds while maintaining strong reliability.

Previously, at Aasma Technology Solutions, I automated Docker and Jenkins pipelines for eight microservices, worked with AWS Glue and S3 integrations, and improved production observability using Sentry and Azure Monitor. These experiences made me realize that I am especially interested in the infrastructure beneath an application—how services are deployed, scaled, monitored, secured, and kept reliable.

What particularly attracts me to Persona is that the company is using technology to make online interactions safer and more trustworthy. Persona helps businesses verify real people and businesses, prevent fraud, and protect legitimate users while respecting their privacy.

---

8. Detailed work-experience preparation

USC Dornsife — Software Engineer

What your resume establishes

You built production services using FastAPI and PostgreSQL for more than 500 users. Your work included CI/CD releases, structured logging, automated validation and performance improvements. You reduced latency from approximately 1.2 seconds to 350 milliseconds while maintaining more than 99% reliability.

You also implemented ETL-style pipelines processing more than 5,000 monthly jobs, added reconciliation checks and cloud-backed storage, and improved incident triage through monitoring hooks and documented deployment processes.

HR-ready description

> At USC Dornsife, I worked as a software engineer on production Python platform services used by more than 500 users. My responsibilities covered more than API development—I also worked on PostgreSQL performance, deployment validation, CI/CD, structured logging and production monitoring.
>
> One of my main contributions was identifying and improving a performance bottleneck that reduced latency from around 1.2 seconds to approximately 350 milliseconds. I made sure the changes were validated through automated checks and monitored after deployment so that the performance improvement did not reduce correctness or reliability.
>
> I also worked on ETL-style pipelines processing more than 5,000 jobs per month. I added reconciliation and monitoring mechanisms that made it easier to identify failed or inconsistent jobs and reduced the time required to investigate production issues.
>
> This experience taught me that building the feature is only part of the work. A production system also needs reliable deployments, observability, validation and clear operational procedures.

Be ready to explain

- What did the platform actually do?

- Who were the 500 users?
- What caused the 1.2-second latency?
- Did you add indexes, optimize queries, remove repeated calls or change application logic?
- How did you measure 350 milliseconds?
- What does “99% reliability” mean?
- What happened when an ETL job failed?
- What did reconciliation check?
- What monitoring information did you collect?
- What did you personally own versus what the team owned?

Best connection to Persona

> This experience is relevant to Persona because it taught me to treat performance, observability and reliability as product requirements rather than afterthoughts.

Grad8 — Software Engineer Intern

What your resume establishes

You shipped Flask APIs on AWS supporting more than 150 users and approximately 10,000 daily requests. You used GitHub Actions for CI/CD and owned deployment validation, documentation and stable production performance.

You also participated in code reviews and pair programming, reduced feedback turnaround by 60%, independently debugged production issues and improved maintainability.

HR-ready description

> At Grad8, I worked on production Flask APIs deployed on AWS. The system supported more than 150 users and handled around 10,000 daily requests.
>
> My work included building API functionality, maintaining the GitHub Actions deployment workflow, validating releases and investigating production issues. Because the team was iterating quickly, one important part of my role was making sure speed did not come at the expense of system stability.
>
> I also participated actively in code reviews and pairing sessions. I focused on making feedback specific and actionable, which helped reduce the turnaround time between an implementation and an approved change.

> The experience improved both my production-debugging skills and my ability to collaborate in a fast-moving engineering environment.

Be ready to explain

- What did the Flask APIs do?
- Which AWS services were used?
- What triggered the GitHub Actions pipeline?
- What stages existed: linting, tests, build, deployment and validation?
- How did you validate that a deployment was healthy?
- Describe one production problem you independently debugged.
- How was the 60% feedback-turnaround improvement calculated?
- What was one code-review suggestion that improved maintainability?

Best connection to Persona

> Persona's infrastructure engineers support teams that need to ship quickly without compromising reliability. Grad8 taught me to balance rapid iteration with validation, documentation and production stability.

Aasma Technology Solutions — Software Development Engineer, SecureSync

What your resume establishes

You worked on cross-platform software supported by Jenkins CI/CD and more than 20 PyTest or Mockito test suites across Linux, Windows and macOS. The work improved pipeline efficiency by 68%.

You also troubleshot AWS Glue and S3 ETL integrations, improved response times by 30% and documented deployment procedures to reduce manual operations and improve infrastructure handoffs.

HR-ready description

> At Aasma, I worked on SecureSync, a cross-platform software system that needed to operate reliably across Linux, Windows and macOS.
>
> I contributed to the Jenkins CI/CD process and created or maintained more than 20 automated test suites using PyTest and Mockito. The objective was to detect platform-specific failures earlier and make releases more repeatable.

I also worked on AWS Glue and S3-based ETL integrations. When we experienced performance or integration problems, I used logs and controlled testing to isolate whether the issue originated in data ingestion, transformation, storage access or the application layer.

Through pipeline improvements and troubleshooting, we improved processing efficiency and response performance while reducing the amount of manual operational work required from the team.

Be ready to explain

- What was SecureSync?
- What made it cross-platform?
- Which components were written in Python versus Java?
- What did the PyTest and Mockito suites test?
- How did you manage OS-specific differences?
- What exactly does “pipeline efficiency improved by 68%” mean?
- Was it runtime, throughput, deployment frequency or manual effort?
- How was AWS Glue used?
- What format was stored in S3?
- What caused the 30% response-time improvement?
- Describe one infrastructure handoff that your documentation improved.

Best connection to Persona

This experience is relevant because infrastructure changes must work consistently across environments, and automated testing is essential before deploying changes that may affect production systems.

Aasma Technology Solutions — Full-Stack Developer Intern, VibrantChem

What your resume establishes

You automated Docker and Jenkins CI/CD pipelines for eight microservices on Azure, reducing deployment time by 50%. You improved observability using Sentry, Azure Monitor and structured logging.

You also built REST APIs and WebSocket synchronization for more than two million records in Azure SQL, with work involving relational schema design, query tuning, reliability and performance.

HR-ready description

> During my earlier internship at Aasma, I worked on VibrantChem, which used a microservices architecture deployed on Azure.
>
> I helped containerize and automate deployment for eight microservices using Docker and Jenkins. Before the automation, deployments required more manual steps and were slower to validate. I worked on making the build and deployment process repeatable, which reduced deployment time by approximately 50%.
>
> I also improved production visibility using structured logs, Sentry and Azure Monitor. That made it easier to determine which service had failed and whether the cause was application logic, deployment configuration or database behavior.
>
> On the backend, I worked on REST APIs and WebSocket-based synchronization involving more than two million Azure SQL records. That required attention to schema design, query efficiency and the reliability of real-time updates.

Be ready to explain

- What did each of the eight microservices do?
- How were they containerized?
- Did every service have its own Dockerfile?
- What were the Jenkins pipeline stages?
- Where were Docker images stored?
- What happened when a deployment failed?
- Why were WebSockets required instead of polling?
- How did you prevent inconsistent updates?
- How did you query two million records efficiently?
- What did Sentry show that ordinary logs did not?
- How was the 50% deployment improvement measured?

Best connection to Persona

> This was the experience that first made me interested in infrastructure because the deployment and observability improvements benefited every engineer working on the system, not only one feature.

LST Capitals — Web Developer Intern

What your resume establishes

You built Flask APIs and Databricks ingestion pipelines for more than 50,000 records with sub-200-millisecond responses. You documented system behavior and collaborated on requirements across more than 12 backend endpoints.

You also diagnosed production issues using logging and monitoring and reduced manual handling by 70%.

HR-ready description

> At LST Capitals, I worked on Flask APIs and data-ingestion pipelines involving more than 50,000 records.
>
> I collaborated with stakeholders to define requirements across more than 12 backend endpoints and worked on keeping API response time below 200 milliseconds. I also documented the expected behavior of the endpoints so integration partners could use them consistently.
>
> When production issues occurred, I relied on logging and monitoring to trace failures through the ingestion and API layers. I then helped automate parts of the process that had previously required manual intervention, reducing manual handling by approximately 70%.
>
> This role gave me early experience translating business requirements into reliable backend behavior while improving the operational side of the system.

Be ready to explain

- What data was being ingested?
- How did Databricks fit into the pipeline?
- What transformations were performed?
- How did you achieve sub-200-millisecond API responses?
- How were the 12 endpoints organized?
- What manual process was reduced?
- How was the 70% improvement measured?
- Describe a production defect and how logs helped identify it.

Best connection to Persona

> LST taught me how to connect data pipelines, APIs, monitoring and documentation—areas that are also important for operating an infrastructure platform.

---

9. Detailed project preparation

Linguistix

What your resume establishes

Linguistix uses Python, AWS ECS/Fargate, Docker and CI/CD. You built a containerized REST API and used AWS CodePipeline with validation gates and production-troubleshooting workflows.

HR-ready description

> Linguistix was a project where I focused on turning a Python REST API into a production-style cloud service.
>
> I packaged the application with Docker and deployed it using AWS ECS with Fargate, which allowed the container to run without directly managing virtual-machine infrastructure. I then created a CodePipeline-based CI/CD workflow so changes could be built, validated and deployed consistently.
>
> I included validation gates so a failed build or test would prevent an unhealthy release from progressing. I also created troubleshooting procedures for deployment and runtime failures.
>
> The main lesson from this project was that production readiness is not only about writing a working API. It also requires reproducible builds, automated deployment, health validation, logs and a process for diagnosing failures.

Add your real product explanation

Your resume describes the infrastructure but not what Linguistix does. Prepare one sentence:

> Linguistix helps **[target user]** solve **[specific language-related problem]** by **[main functionality]**.

Do not enter the interview without this sentence.

Technical follow-ups

- Why ECS/Fargate instead of EC2?
- Why Docker?
- What did the Dockerfile contain?
- What triggered CodePipeline?

- What were the pipeline stages?
- What was a validation gate?
- How were environment variables and secrets managed?
- How were logs collected?
- What happened when a new deployment failed?
- Did the service use a load balancer or health checks?

FinNexus

What your resume establishes

FinNexus uses Next.js, TypeScript, Kubernetes and WebSockets. Your resume describes a platform designed for more than 50,000 users and one million transactions, with real-time updates, secure access controls and performance-tuned backend services.

HR-ready description

> FinNexus was a real-time platform where I focused on scalability, secure access and live data updates.
>
> The frontend was built using Next.js and TypeScript, while the deployment architecture used Kubernetes. WebSockets enabled the server to push updates to connected users instead of requiring continuous polling.
>
> I designed the system with high transaction volume in mind, including secure access control, performance-aware backend services and container orchestration for scaling.
>
> The project gave me practical experience thinking about how application design, networking, authentication and infrastructure work together under higher load.

Critical metric warning

Be completely clear whether:

- 50,000 users were actual production users,
- registered test users,
- a design capacity,
- or a load-test result.

Do the same for one million transactions.

A safe version when these were load-tested or simulated is:

> I designed and load-tested the platform for scenarios representing 50,000 users and one million transactions.

Do not imply real production usage unless it was real.

Technical follow-ups

- What kind of transactions did the system process?
- Which database was used?
- How did authentication work?
- How were authorization rules enforced?
- How were WebSocket connections scaled?
- Did you use a message broker or pub/sub?
- How did Kubernetes distribute traffic?
- What were the deployment, service and ingress configurations?
- How did you prevent duplicate or out-of-order updates?
- How were the performance numbers measured?
- What was the largest actual load test?

CPU-Scheduling Simulator

What your resume establishes

You implemented performance-critical scheduling logic in C++ with a React interface. The simulator handled edge cases and evaluated resource allocation across more than 1,000 simulated process combinations.

HR-ready description

> The CPU-Scheduling Simulator was a systems project that allowed users to enter processes and compare how different scheduling strategies allocate CPU time.
>
> I implemented the core scheduling logic in C++ and connected it to a React interface that visualized scheduling behavior and performance metrics.
>
> The most challenging part was handling edge cases such as simultaneous arrival times, idle CPU periods, priority ordering and time-quantum behavior. I tested the algorithms across more than 1,000 process combinations to verify correctness.
>
> This project strengthened my understanding of operating-system fundamentals, resource allocation and how scheduling decisions affect throughput, latency, fairness and responsiveness.

Only mention algorithms you actually implemented

Potential algorithms may include:

- First Come, First Served
- Shortest Job First
- Shortest Remaining Time
- Priority scheduling
- Round Robin

Confirm the actual list before discussing it.

Technical follow-ups

- Which scheduling algorithms did you implement?
- What is preemptive versus non-preemptive scheduling?
- How did you calculate waiting time?
- How did you calculate turnaround time?
- How did you handle ties?
- Why did you use C++?
- How did React communicate with the scheduling logic?
- What edge case was most difficult?
- How did you generate the 1,000 test combinations?
- What trade-off does Round Robin's time quantum create?

Persona connection

> Although it was an academic-style project, it gave me a stronger systems foundation for understanding resource allocation, performance and operating-system behavior, which is useful for infrastructure engineering.

---

10. Which experience to use for different recruiter questions

"What accomplishment are you most proud of?"

Use **USC Dornsife**:

> I'm most proud of improving the latency of a production platform service from approximately 1.2 seconds to 350 milliseconds while maintaining reliability. I liked this accomplishment because it required understanding both the application and database behavior, validating the change and monitoring it after deployment.

"Tell me about an infrastructure project"

Use **Aasma VibrantChem**:

> I automated Docker and Jenkins deployment pipelines for eight Azure-hosted microservices and improved production observability through structured logging, Sentry and Azure Monitor.

"Tell me about production debugging"

Use **Grad8**, **USC Dornsife** or **LST**, based on the incident you remember most clearly.

"Tell me about working across platforms"

Use **SecureSync**:

> I supported software and automated testing across Linux, Windows and macOS using Jenkins, PyTest and Mockito.

"Tell me about scalability"

Use **FinNexus**, but explain whether the scale came from real traffic, simulation or load testing.

"Tell me about systems fundamentals"

Use the **CPU-Scheduling Simulator**.

"Tell me about cloud deployment"

Use **Linguistix**:

> Docker, ECS/Fargate, CodePipeline, validation gates and production troubleshooting.